Packages
and
GUI Programming with Swing

Building Desktop Applications in Java

What We Will Learn

- Understanding Packages
- Understand the difference between AWT and Swing.
- Master the Swing Component Hierarchy.
- Create windows using JFrame and panels using JPanel.
- Utilize Layout Managers to organize components.
- Implement Event Handling (ActionListeners).
- Understand the Event Dispatch Thread (EDT).
- Build a functional desktop application.

Understanding Java Packages

- A namespace that organizes a set of related classes and interfaces.
- **Analogy:** Think of packages as **Folders** on your computer.
- **Purpose:**
  - Prevents naming conflicts.
  - Controls access (encapsulation).
  - Makes code easier to maintain.
- **Declaration:** Must be the **first line** of your Java file.

```
package com.mycompany.guiapp;
```

Avoiding Naming Conflicts

- Java has thousands of built-in classes. Sometimes they have the same name.
- Classic Example: List
  - java.awt.List (AWT GUI component)
  - java.util.List (Collection interface)
- Without Packages/Imports: The compiler doesn't know which List you mean.
- Solution: Fully qualified names or specific imports

```
// Ambiguous without imports
List myList = new List();

// Clear with imports
import java.util.List;
import java.awt.List; // Cannot import both with wildcards!
```

The import Statement

- Allows you to use classes without typing the full package name.
- Specific Import (Recommended)

```
import javax.swing.JFrame;
import javax.swing.JButton;
```

- **Wildcard Import (Use with Caution):**

```
import javax.swing.*;
```

- **Rule:** import statements come **after** package and **before** class.

GUI: History: AWT vs. Swing

- **AWT (Abstract Window Toolkit):**
  - Java 1.0.
  - **Heavyweight:** Uses native OS components (buttons look like Windows buttons on Windows, Mac buttons on Mac).
  - Limited components, prone to bugs across platforms.
- **Swing:**
  - Java 1.2 (part of JFC - Java Foundation Classes).
  - **Lightweight:** Written entirely in Java; draws its own pixels.
  - **Pluggable Look & Feel:** Can mimic native OS or use a custom style.
  - Richer set of components (JTable, JTree, JTabbedPane).

Controlling Access (Encapsulation)

- Packages work closely with access modifiers.
- **public**: Accessible from any other class.
- **private**: Accessible only within the same class.
- **protected**: Accessible within the package and subclasses.
- **Default (no modifier)**: Accessible **only within the same package**.
- **GUI Best Practice:**
  - Make UI components private.
  - Provide public methods to update them.

```
public class LoginForm {
    private JTextField passwordField;
    public void clearForm() { ... }
}
```

The Swing Architecture

- **java.awt.Component**: The root abstract class.
- **java.awt.Container**: A component that can hold other components.
- **javax.swing.JComponent**: The base class for all Swing components.
  - Adds double buffering and keyboard handling.
- **Top-Level Containers:**
  - JFrame (Window with borders/title).
  - JDialog (Popup window).
  - JApplet (Deprecated, web-based).

Creating a Window (JFrame)

- JFrame represents the main application window.
- **Key Setup Steps:**
  - Instantiate the frame.
  - Set default close operation.
  - Set size (or pack()).
  - Set visibility to true.
  - **Always start on the EDT.**

```
JFrame frame = new JFrame("My App");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setSize(400, 300);
frame.setVisible(true);
```

Common Swing Components

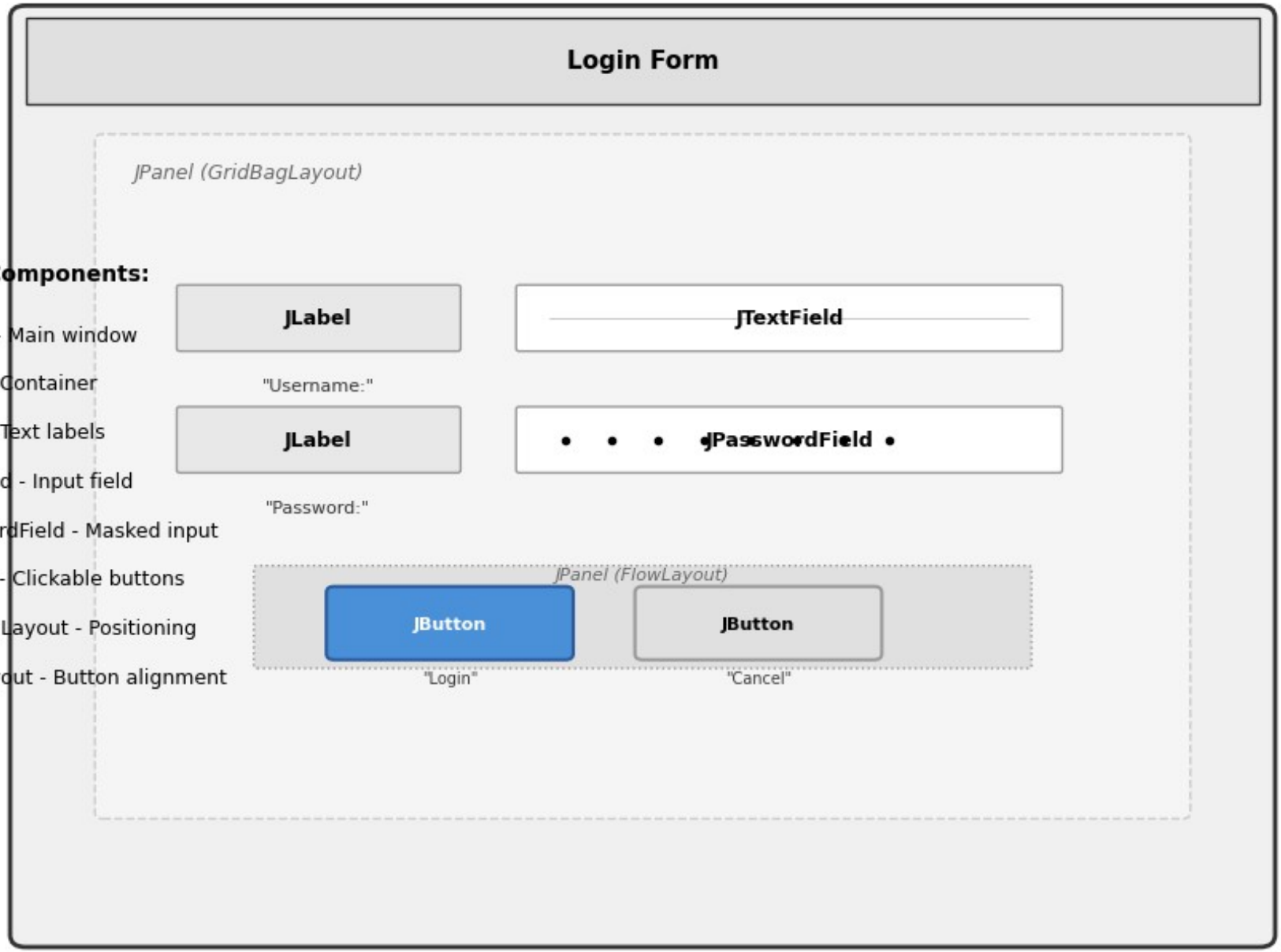
- All start with **'J'**.
- **JLabel**: Display text or images.
- **JButton**: Clickable button.
- **JTextField**: Single-line text input.
- **JTextArea**: Multi-line text input (wrap in JScrollPane).
- **JCheckBox / JRadioButton**: Selection controls.

```
JLabel label = new JLabel("Username:");
JTextField text = new JTextField(20);
JButton btn = new JButton("Submit");
```

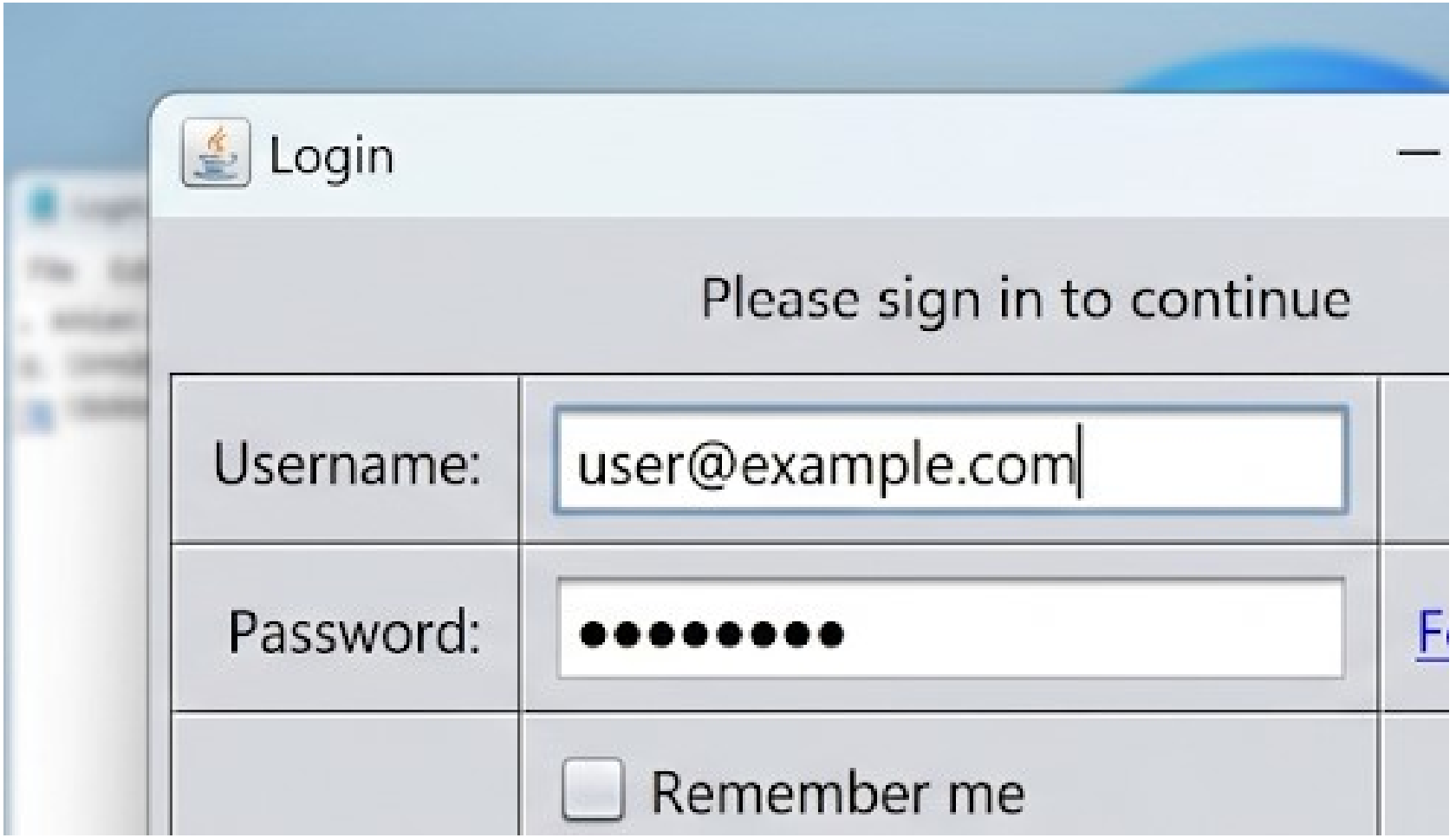
Swing Login Form component Layout.

Swing Components:

- JFrame - Main window
- JPanel - Container
- JLabel - Text labels
- JTextField - Input field
- JPasswordField - Masked input
- JButton - Clickable buttons
- GridBagLayout - Positioning
- FlowLayout - Button alignment



Screenshot of a simple login form showing swing components.



Managing Component Placement

- **Why?** Hard-coding coordinates (setBounds) is bad practice (doesn't scale, ignores OS font sizes).
- **Layout Managers** automatically arrange components.
- **Common Layouts:**
  - **BorderLayout**: North, South, East, West, Center (Default for JFrame).
  - **FlowLayout**: Left-to-right, wraps lines (Default for JPanel).
  - **GridLayout**: Grid of equal-sized cells.
  - **BoxLayout**: Single row or column.

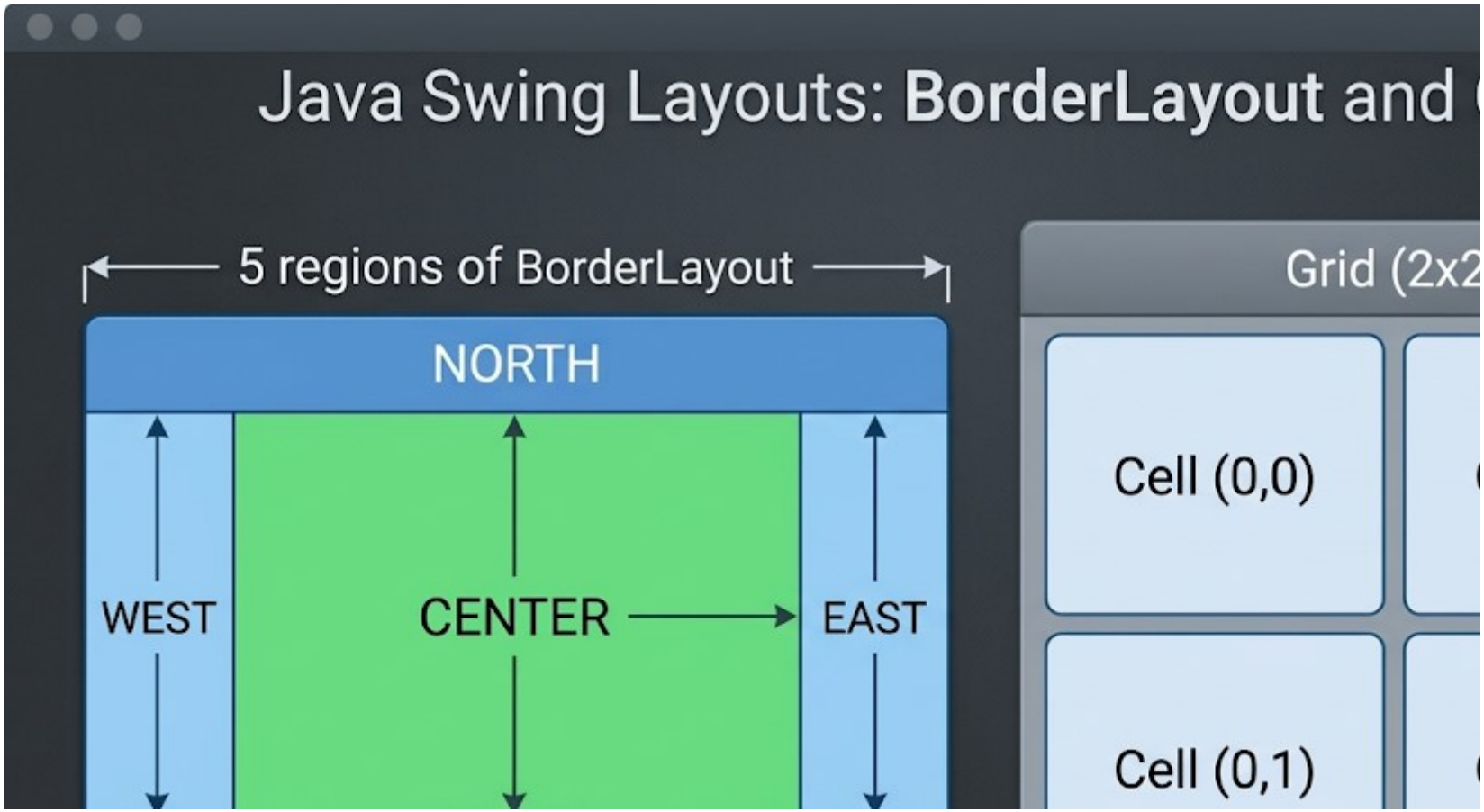
Layout Manager

- **BorderLayout:**java

```
frame.setLayout(new BorderLayout());
frame.add(new JButton("North"),
  BorderLayout.NORTH);
frame.add(new JButton("Center"),
  BorderLayout.CENTER);
```

- **GridLayout:**java
  - `JPanel panel = new JPanel(new GridLayout(2, 2)); // 2 rows, 2 cols`
  - `panel.add(new JButton("1"));`
  - `panel.add(new JButton("2"));`
- **Nested Layouts:**
  - Combine panels with different layouts to create complex UIs.

Diagram showing the 5 regions of BorderLayout and a 2x2 grid.



The Delegation Event Model

- **How Events Work**
  - **Source:** The component (e.g., JButton).
  - **Event:** An object describing the change (e.g., ActionEvent).
  - **Listener:** An interface that waits for the event (e.g., ActionListener).
  - **Registration:** component.addListener(this)

Flow:

- User clicks button.
- Button creates ActionEvent.
- Button notifies registered ActionListener.
- actionPerformed method executes.

Implementing Listeners

- Handling Button Clicks
  - **Option 1:** Class implements ActionListener.
  - **Option 2:** Anonymous Inner Class (Common for simple UIs).
  - **Option 3:** Lambda Expression (Java 8+).

```
// Lambda Approach
button.addActionListener(e -> {
    String text = textField.getText();
    label.setText("Hello " + text);
});
```

Menus & Standard Dialogs

- Menus: JMenuBar, JMenu, JMenuItem.
  - Added to JFrame via setJMenuBar().
- Dialogs: JOptionPane.
  - Quick popups for messages or input.

```
JOptionPane.showMessageDialog(frame, "Save Successful!");
String input = JOptionPane.showInputDialog("Enter Name:");
int confirm = JOptionPane.showConfirmDialog(frame, "Exit?");
```

Project Structure

Organizing a GUI Project

- Don't put all classes in the default package.
- **Recommended Structure**

```
src/
└── com/
    └── myapp/
        ├── Main.java      (Entry point)
        ├── view/          (JFrames, JPanels)
        ├── controller/    (ActionListeners, Logic)
        └── model/         (Data classes)
```

- **Benefits**
  - ✓ Separates UI from Logic (MVC Pattern).
  - ✓ Easier to scale as the app grows.

Common Mistakes

Package Pitfalls to Avoid

- ✕ **The Default Package:**
  - Never leave out the package declaration in serious projects.
  - Classes in the default package cannot be imported by classes in other packages.
- ✕ **Naming Conflicts:**
  - Do not name your class String, System, or Frame.
- ✕ **Circular Dependencies:**
  - Package A imports Package B, and Package B imports Package A.
- ☑ **Best Practice:**
  - Use lowercase for package names (e.g., com.example.app).
  - Use CamelCase for class names.

The Event Dispatch Thread (EDT)

- Thread Safety in Swing
  - Swing is NOT thread-safe.
  - All UI updates must happen on the Event Dispatch Thread.
  - Problem: Long-running tasks (DB calls, downloads) freeze the UI if done on EDT.
  - Solution:
    - Start UI on EDT: SwingUtilities.invokeLater(...).
    - Run heavy tasks in background threads (SwingWorker).

```
SwingUtilities.invokeLater(() -> {
    new MyApp().createAndShowGUI();
});
```

Pluggable Look and Feel

- Swing allows changing the UI skin at runtime.
- **System Look:** Matches the OS.
- **Nimbus:** Modern Java look (included in JDK).
- **CrossPlatform:** Java's default (Metal).

```
try {
    UIManager.setLookAndFeel(
        UIManager.getSystemLookAndFeelClassName());
} catch (Exception e) { e.printStackTrace(); }
```

- Java's Must be set before creating any components.

Putting It All Together

- Task: Create a Temperature Converter.
- Requirements:
  1. JFrame titled "Converter".
  2. JTextField for input.
  3. Two JButtons (C to F, F to C).
  4. JLabel to show result.
  5. Use BorderLayout.
  6. Handle invalid input (try-catch).
  7. Launch on EDT.

Summary & Next Steps

- **Summary**:
  - Swing is lightweight and mature.
  - Use Layout Managers, not absolute positioning.
  - Respect the Event Dispatch Thread.
  - Use Listeners for interaction.
- **Modern Alternatives**:
  - JavaFX: Newer standard for Java desktop.
  - Web Technologies: Electron, React, etc. (Industry trend).
- **Resources**:
  - Oracle Swing Tutorial docs.
  - "Core Java Volume II" (Horstmann).

Assignment 1

https://classroom.github.com/a/4FKAK09X